WEEK 3 GROUP EXERCISE AND ASSESSMENT

Agile Methodologies: From Client Problem to Agile Development Plan

Mode	**Group Activity**
Assessment Period	Week 3
Submission Deadline	September 25, 2026 — 12:00 midnight
Presentation	During the lecture class next week

I. Purpose

This activity requires each group to apply Agile concepts directly to the application selected for development with its actual client or stakeholder. Using the identified client problem and proposed application, the group will transform the project idea into an initial Agile development plan. The group will identify user needs, organize requirements into epics and user stories, create and prioritize a Product Backlog, select an appropriate Agile approach, and plan the first development iteration.

The purpose is not to complete the application this week. The purpose is to establish a clear, realistic, and collaborative process for developing the application incrementally while incorporating client feedback.

II. Group Exercises

Exercise 1 — Understand the Client Problem

Review the information gathered from your client and complete the following.

Project/Application Title: *RenTahanan*: RenTahan: A Bedspace Occupancy and Rent Management System

Client/Organization: Evelyn Lady's Dormitory

Primary Users:

- Boarding House Owners / Landlords/ Property Caretakers (Primary Administrators)
- Student Boarders / Tenants (End Users)

Client Problem / Problem Statement:

Managing large-scale student dormitories (50+ bed spaces) using physical paper notebooks and manual paper contracts creates three primary operational bottlenecks.

- **Inflexible Rent Billing Administration:** Existing commercial systems often assume submeter calculations, but real-world operations vary. Evelyn's Lady's Dormitory operates on flat-rate monthly rents with bundled utilities (laptops, phones, and rice cookers are included free), requiring a system tailored to fixed billing rather than complex meter-splitting engines.
- **Paper-Based Occupancy Tracking Across Multiple Room Types:** Caretakers track vacancies across 50+ bed spaces – categorized into single (good for 1), double (good for 2), and quad (good for 4) rooms by manually scanning handwritten tables or lists, leading to slow responses during student inquiries and potential room/bed assignment errors.
- **Lack of Role-Based Task Separation:** Because the property owner handles higher-level financial calculations and record auditing while the on-site caretakers handles cash collection and OR issuing, a paper logbook fails to provide separate, secure access views tailored to each role's responsibilities.
- **Disjointed Security Deposits and Contract Tracking:** Advance payments (1-month advance and 1-month deposit) and lease start/end dates are kept in separate physical paper contract folders, making move-out clearance slow and reliant on manually pulling paper documents to check for damage deductions or unpaid dues.
- **Audit and Receipt Verification Vulnerabilities:** Recording transactions solely through handwritten paper official receipts (ORs) and notebook log lines increase the risk of lost receipt records, human transcription errors, and missing signatures during audit reconciliations between caretakers and owners.

Current Process:

- **Record-Keeping and Occupancy Tracking:** Caretakers maintain physical paper notebook structured as clustered room tables. Each table (e.g., Room 1) lists rows of tenant names alongside columns for Amount Paid, Date, OR Number, Tenant Signature, and Remarks. Bed Vacancies across single, double, and quad rooms are checked manually by flipping through and reading these handwritten tables.
- **Billing and Utilities:** The property owner establishes fixed monthly rent rates that include all standard utilities (electricity, water, and basic appliances like laptops, and rice cookers). Dues are checked manually against paper ledger rows rather than through automated digital notifications.

- **Payments and Receipt Collection:** Tenants are required to pay in full per month (no partial micro-installments or down payments are allowed). New boarders must pay 1 month advance payment and 1 month advance security deposit upon moving in.
- **Receipting and Verification:** When a tenant pays, the caretaker manually writes a physical paper Official Receipt (OR) and hands it to the student. The caretaker then logs the Date and OR number into the paper notebook table and requires the tenant to sign the ledger row directly.
- **Lease and Deposit Storage:** Physical signed paper contracts are kept in paper document folders in the office. Caretakers manually locate and read these contract documents when a tenant moves out to verify advance deposits and lease terms.

Proposed Application: A mobile-first web application designed to digitize boarding house operation through:

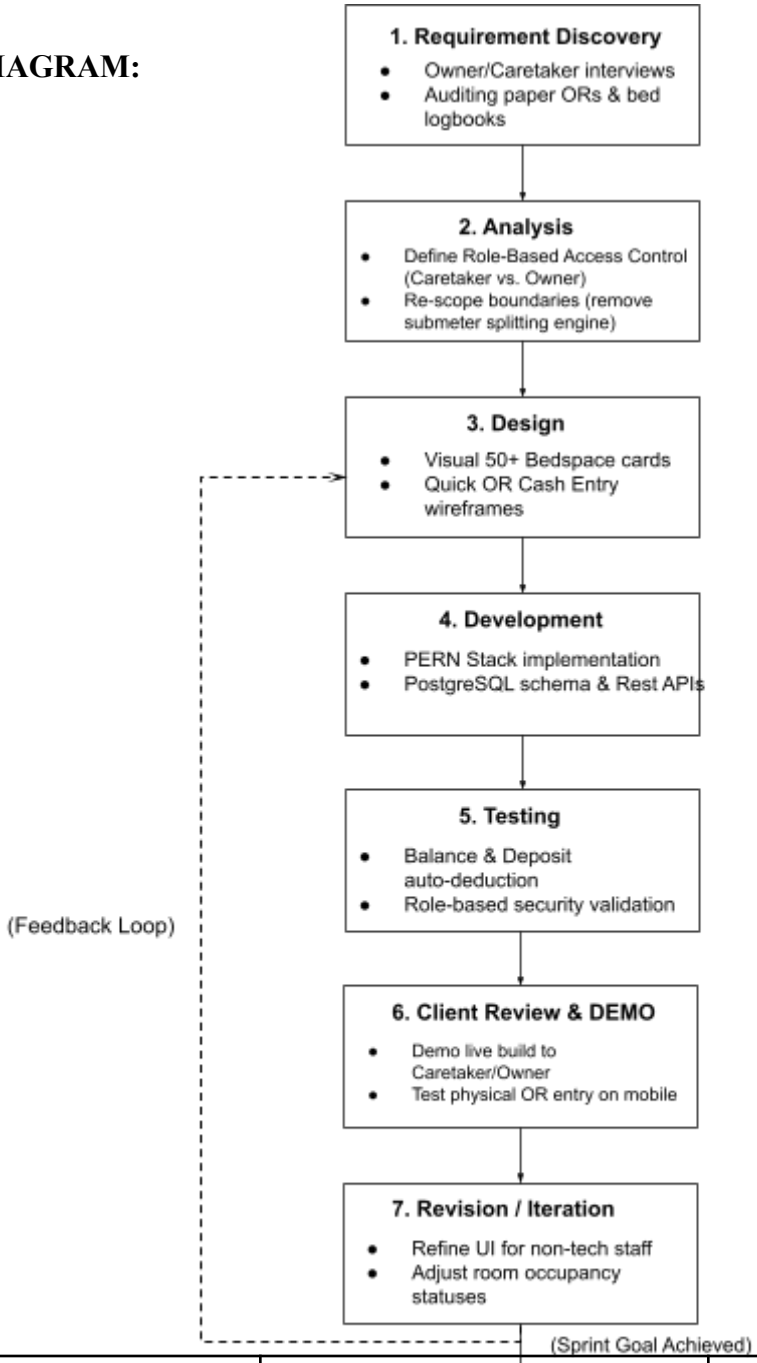
- **Visual Property and Bed-Space Occupancy Map:** A dynamic, color-coded dashboard displaying real-time bed and room availability (Single, Double , Quad layouts) with instant status cards (Paid, Overdue, Vacant Bed) to replace manual paper lists.
- **Configurable Rent and Utility Billing Module:** A flexible management engine supporting both fixed-rent bundled utilities and submetered cost-splitting, allowing property owners to define customized pricing structures and included appliance policies.
- **Digital Ledger with Caretaker OR Logging:** A role-based workflow enabling caretakers to quickly record cash collections, input physical Official Receipt (OR) reference numbers, and auto-update tenant ledgers from a smartphone screen.
- **Automated Debt Alerts and Departure Clearance:** An integrated tracking system that flags overdue accounts, logs 1-month advance and deposit balances, and generates move-out clearance reports to prevent tenants from leaving behind uncollected debts.
- **Contract and Lease Lifecycle Digitalization:** A secure database module to store digital lease terms, emergency contacts, and deposit records directly attached to tenant profiles for instant lookup.

Exercise 2 — Map the Application Development Life Cycle

Create a visual development life-cycle diagram showing how your team will move from understanding the client's requirements to final release. Your diagram must show iteration and feedback rather than a purely one-way process.

Minimum flow: Requirement Discovery → Analysis → Design → Development → Testing → Client Review/Feedback → Revision/Iteration → Release

FOR THE DIAGRAM:



Stage	What Will Your Team Do?	Expected Output
Requirement Discovery	Conduct on-site interviews with the caretaker and examine existing paper logbooks, clustered tables, and contract sheets.	Transcribed interview data and photo-audited paper workflow artifacts.

Stage	What Will Your Team Do?	Expected Output
Analysis	Re-scope system boundaries by removing submeter splitting engines; define Role-Based Access Control (Keeper vs. Owner) and 50+ bed tracking rules.	Refined Software Requirements Specification (SRS) and normalized database schema.
Design	Draft mobile-first wireframes and high-contrast UI mockups emphasizing quick OR input fields, high-visibility bed status cards (Single/Double/Quad), and simplified payment forms for non-tech-savvy caretakers.	High-fidelity Figma prototypes, UI design kit, and component layout wireframes for mobile and desktop screens.
Development	Build core React frontend views and Express/Node.js REST APIs connected to a PostgreSQL database (PERN stack), implementing secure JWT authentication, bed occupancy logic, and digital receipt/ledger tracking.	Functional, testable Minimum Viable Product (MVP) running on local environments and connected to a live database.
Testing	Execute unit tests for balance calculations and end-to-end testing for caretaker payment entry workflows, verifying database transaction safety and OR reference number logging.	Bug tracking logs, test coverage scripts, and pass/fail execution reports verifying role-based route security.
Client Review	Walk the caretaker and owner of Evelyn Lady's Dormitory through the live MVP on a mobile device to observe real-world usability and collect	Usability observation notes, client feedback documentation, and a prioritized list of user change requests.

Stage	What Will Your Team Do?	Expected Output
	feedback on logging payments and checking bed statuses.	
Revision / Iteration	Refactor the UI based on caretaker feedback (e.g., larger keypad buttons, faster room filtering, clearer contract deposit indicators)/	Prioritized Iteration Backlog and updated software patch. Updated backlog items, refined React UI components, and a technical debt patch plan.
Release	Deploy the production application onto live cloud hosting (Vercel/Render/Supabase) and onboard Evelyn Lady's Dormitory with their actual 50+ bed data and active tenant records.	Live application URL, populated database instance, and user quick-start guides for both caretaker and owner roles.

Exercise 3 — Compare Sequential and Iterative Development

1. What problems might occur if the client changes a requirement after development has already started?
   - **Rework on Financial & Ledger Logic**: Rebuilding payment tracking workflows mid-way forces developers to discard completed code and restructure database models handling Official Receipts (ORs) and balances.
   - **Wasted Development Effort**: Spending time building rigid metering tools before confirming the boarding house uses flat-rate bundled rent results in discarded features and lost hours.
   - **Disrupted Role- Based Permissions**: Changing how tasks are divided between owners and caretakers late in development breaks security access controls and user interface layouts.
   - **Database Schema Migrations:** Altering how 1-month advance payment, security deposits, or multi-bed room layouts (single, double , quad) are structured breaks foreign key constraints in PostgreSQL, delaying sprint schedules.
2. Which parts of your application are most likely to change after the client sees the first working version?

- **Role-Based Views & Dashboard Layouts:** Adjustments to simplify screen layouts so caretakers can quickly enter cash payments while owners get dedicated auditing tools.
- **Occupancy & Bedspace Visual Maps:** Tweaks to how single, double, and quad room vacancies are displayed to ensure fast checking during student inquiries.
- **Official Receipt (OR) & Payment Entry Forms:** Updates to match the exact paper-logging sequence caretakers use when logging payments and student signatures.
- **Move-Out Clearance Workflow:** Adjustments to how advance deposits and contract dates are pulled to verify unpaid balances during student departures.

3. How can iterative development help your team respond to these changes?
    - **Incremental Delivery**: Delivering core modules first (e.g., bedspace occupancy tracking) lets caretakers test features while developers build the payment logging engine in the next sprint.
    - **Flexible Feature Adjustments**: New requests—such as adding flat-rate appliance surcharges—can easily be added to upcoming iterations without halting the entire project.
    - **Early Risk Mitigation:** Testing role-based permissions early ensures caretakers and owners get the exact tools they need without requiring major overhauls later.
4. Why is frequent client feedback important for your particular application?

    - **Supports Real-World Role Division:** Frequent feedback ensures the system mirrors actual operations, where caretakers handle cash/ORs and owners conduct financial audits.
    - **Prevents Financial & Deposit Leakage:** Direct input helps eliminate manual calculation errors, preventing students from moving out with unpaid dues.
    - **Ensures Usability for Non-Tech Users:** Regular testing guarantees that staff transitioning from paper logbooks find the digital interface clear and easy to navigate.

Group conclusion: In 3–5 sentences, explain why an iterative approach is appropriate (or not appropriate) for your project.

An iterative approach (Agile/Scrum) is essential for RenTahanan because our target client operates on informal, paper-based routines that cannot be fully anticipated in a single upfront specification. Developing in short, testable cycles allowed our team to quickly pivot away from submeter splitting after learning the client uses flat-rate bundled rent. Delivering small increments such as visual bed tracking and caretaker receipt logging guarantees that non-tech-savvy staff can thoroughly test mobile usability before full production deployment.

Exercise 4 — Convert Client Problems into Epics and User Stories

Identify at least 3 initial Epics and create user stories based on actual client/user needs.

Use the format: As a [type of user], I want [goal/function], so that [reason/value].

ID	Epic	User Story
US-01	Occupancy & Bedspace Management	As a landlord, I want to view my property as color-coded room/bed cards, so that I can instantly see which beds are occupied, overdue, or vacant at a glance.
US-02	Bedspace Assignment and Tenant Registration	As a caretaker, I want to assign new boarders to specific bed spaces and register their contract details, so that i can prevent double-bookings and maintain accurate bed occupancy records.
US-03	Tenant Portal Lease and Record Verification	As a student boarder, I want to view my assigned room number and check-in contract details online, so that i can verify my agreement without checking paper documents.
US-04	Fixed-Rate Rent & All-Inclusive Utility Allocation	As a landlord, I want the system to generate fixed monthly rent statements with bundled utility allowances, so that I can send consistent monthly bills without calculating electric or water meters.
~~US-05~~	~~High-Power Appliance Flat-Rate Adjustments~~	~~As a caretaker, I want to add fixed surcharges for extra high-power appliances brought~~

ID	Epic	User Story
		~~by boarders (e.g., personal mini-fridges, air conditioning, induction cookers), so that electricity costs are covered fairly.~~
US-06	Tenant Itemized Statement View	As a student boarder, I want to see a clear breakdown of my monthly rent and additional appliance fees, so that I can understand my total bill before paying.
US-07	Payment Logging with Official Receipt (OR) Tracking	As a caretaker, I want to input cash payments along with their Official receipt (OR) numbers, so that every payment is digitally logged with proof.
US-08	Partial Balance & Unpaid Rent Alerts	As a landlord, I want the system to automatically calculate and flag unpaid rent balances, so that I can track remaining debts and prevent revenue loss.
US-09	Digital Receipt & Signature Logging	As a caretaker, I want the system to generate a digital receipt with tenant signature confirmation upon payment, so that both parties have a clear financial record.
US-10	Move-Out Clearance & Contract Deposit Reconciliation	As a student boarder, I want to check my outstanding balance and payment history on my portal, **so that** my advance deposit can be accurately

ID	Epic	User Story
		applied to my final bill when I move out.
US-11	Role-Based Caretaker Cash Entry vs. Landlord Ledger Review	As a caretaker, I want a restricted transaction entry portal that only permits cash collection and OR logging, so that sensitive landlord financial configurations remain secure.
US-12	In-Person Digital Acknowledgment / Signature Logging	As a caretaker, I want to capture a digital signature or manual payment acknowledgement flag when issuing an Official Receipt (OR), so that on-site payments are recorded with mutual proof.
US-13	Evasion Flagging	As a landlord, I want the system to flag past tenants who left with unsettled debts, so that they are prevented from re-registering or renting bedspaces in the future.

Exercise 5 — Write Acceptance Criteria

For each of the 10 user stories, provide basic acceptance criteria describing the conditions that must be satisfied before the story can be considered acceptable. You may use bullet conditions or Given–When–Then.

US-01: Occupancy & Bedspace Management

- Given the landlord or caretaker accesses the dashboard, when they view the room management screen, then the system displays all rooms and bed capacities (e.g., solo, good for 2, good for 4) with color-coded status badges for vacant, occupied, or full beds (e.g., Green = Occupied, Blue = Vacant, Red = Overdue).

US-02: Bedspace Assignment and Tenant Registration

- Given a new boarder checks in and submits their 1-month advance rent and 1-month deposit, when the caretaker assigns them to an open bed in a specific room, then the system updates that bed to "Occupied" and links the tenant profile and signed contract record to the room.

US-03: Tenant Portal Lease and Record Verification

- Given a student boarder logs into their account, when they navigate to their profile page, then they can view their assigned room number, check-in date, fixed monthly rent rate, and advance deposit history.

US-04: Fixed-Rate Rent & All-Inclusive Utility Allocation

- Given the owner or caretaker generates monthly billing for rooms with fixed or all-inclusive rent structures, when the monthly billing cycle triggers, then the system applies the fixed monthly rate per boarder including all bundled utility permissions (laptops, phones, appliances) without requiring submeter calculations.

~~**US-05: High-Power Appliance Flat-Rate Adjustments**~~

- ~~Given a boarder brings additional non-standard appliances into a room, when the caretaker adds an extra appliance fee flag to the tenant's profile, then the system automatically adds the fixed surcharge to the tenant's total monthly statement.~~

US-06: Tenant Itemized Statement View

- Given a boarder checks their monthly invoice, when they open the bill details section, then the system displays a breakdown showing total rent due, bundled utilities included, and any logged extra charges.

US-07: Payment Logging with Official Receipt (OR) Tracking

- Given a boarder pays their rent in cash to the caretaker, when the caretaker enters the payment amount, date, and issued Official Receipt (OR) number into the system, then the system logs the entry into the digital ledger, updates the balance, and attaches the OR reference number.

US-08: Partial Balance & Unpaid Rent Alerts

- Given a boarder makes an incomplete payment or misses a monthly due date, when the caretaker views the room ledger, then the system flags the tenant as "Overdue/Has Balance," calculates the exact remaining balance, and prevents hidden unpaid balances.

US-09: Digital Receipt & Signature Logging

- Given a payment transaction is successfully recorded by the caretaker, when the receipt confirmation is submitted, then the system generates a printable or downloadable digital receipt containing the OR number, amount paid, remaining balance, and tenant signature confirmation.

US-10: Move-Out Clearance and Contract Deposit Reconciliation

- Given a student is moving out at the end of the semester, when the caretaker initiates the check-out process, then the system cross-checks all historical unpaid balances and contract terms, automatically deducting owed amounts from the 1-month deposit before issuing final clearance.

US-11: Role-Based Caretaker Cash Entry vs LandLord Review

- Given a caretaker logs into the application, when they navigate to the payment portal, then the system grants access only to cash collection and OR logging interfaces, restricting access to system-wide ledger configurations reserved for the landlord.

US-12: In-Person Digital Acknowledgement / Signature Logging

- Given a caretaker receives an in-person cash payment, when they record the OR details and present the receipt confirmation screen, then the system requires a tenant digital signature or acknowledgement check mark before finalizing the transaction entry into the ledger.

US-13: Evasion Flagging

- Given a tenant attempts to register or check out, when the system cross-checks their account history against unpaid balance records, then it automatically flags any prior unpaid debts ("gansi") to prevent unresolved lease departures or re-registration.

Exercise 6 — Create the Initial Product Backlog

Priority	ID	Epic	User Story / Requirement	Basic Acceptance Criteria
Must Have	US-01	Occupancy & Bedspace Management	**As a** landlord, **I want** to view my property as color-coded room/bed cards, **so that** I can instantly see which beds are occupied, overdue, or vacant at a glance.	Given the landlord or caretaker accesses the dashboard, when they view the room management screen, then the system displays all rooms and bed capacities (solo, good for 2, good for 4) with color-coded status badges for vacant, occupied, or full beds.
Must Have	US-02	Bedspace Assignment and Tenant Registration	**As a** caretaker, **I want** to assign new boarders to specific bed spaces and register their contract details, **so that** I can prevent double-bookings and maintain accurate bed occupancy records.	Given a new boarder checks in and submits their 1-month advance rent and 1-month deposit, when the caretaker assigns them to an open bed in a specific room, then the system updates that bed to "Occupied" and links the tenant profile and signed contract record to the room.
Should Have	US-03	Tenant Portal Lease and Record Verification	**As a** student boarder, **I want** to view my assigned room number and check-in contract details online, **so**	Given a student boarder logs into their account, when they navigate to their profile page, then

Priority	ID	Epic	User Story / Requirement	Basic Acceptance Criteria
			that I can verify my agreement without checking paper documents.	they can view their assigned room number, check-in date, fixed monthly rent rate, and advance deposit history.
Must Have	US-04	Fixed-Rate Rent & All-Inclusive Utility Allocation	**As a** landlord, **I want** the system to generate fixed monthly rent statements with bundled utility allowances, **so that** I can send consistent monthly bills without calculating electric or water meters.	Given the owner or caretaker generates monthly billing for rooms with fixed or all-inclusive rent structures, when the monthly billing cycle triggers, then the system applies the fixed monthly rate per boarder including all bundled utility permissions (laptops, phones, appliances) without requiring meter calculations.
~~**Should Have**~~	~~US-05~~	~~High-Power Appliance Flat-Rate Adjustments~~	~~**As a** caretaker, **I want** to add fixed surcharges for extra high-power appliances brought by boarders, **so that** electricity costs are covered fairly.~~	~~Given a boarder brings additional non-standard appliances into a room, when the caretaker adds an extra appliance fee flag to the tenant's profile, then the system automatically adds the fixed surcharge to the~~

Priority	ID	Epic	User Story / Requirement	Basic Acceptance Criteria
				~~tenant's total monthly statement.~~
Should Have	US-06	Tenant Itemized Statement View	**As a** student boarder, **I want to** see a clear breakdown of my monthly rent and additional appliance fees, **so that** I can understand my total bill before paying.	Given a boarder checks their monthly invoice, when they open the bill details section, then the system displays a breakdown showing total rent due, bundled utilities included, and any logged extra charges.
Must Have	US-07	Payment Logging with Official Receipt (OR) Tracking	**As a** caretaker, **I want** to input cash payments along with their Official Receipt (OR) numbers, **so that** every payment is digitally logged with proof.	Given a boarder pays their rent in cash to the caretaker, when the caretaker enters the payment amount, date, and issued Official Receipt (OR) number into the system, then the system logs the entry into the digital ledger, updates the balance, and attaches the OR reference number
Must Have	US-08	Partial Balance & Unpaid Rent Alerts	**As a** landlord, **I want** the system to automatically calculate and flag unpaid rent balances, **so that** I can track remaining debts and prevent revenue loss.	Given a boarder makes an incomplete payment or misses a monthly due date, when the caretaker views the room ledger, then the system flags

Priority	ID	Epic	User Story / Requirement	Basic Acceptance Criteria
				the tenant as "Overdue/Has Balance," calculates the exact remaining balance, and prevents hidden unpaid balances
Could Have	US-09	Digital Receipt & Signature Logging	**As a** caretaker, **I want** the system to generate a digital receipt with tenant signature confirmation upon payment, **so that** both parties have a clear financial record.	Given a payment transaction is successfully recorded by the caretaker, when the receipt confirmation is submitted, then the system generates a printable or downloadable digital receipt containing the OR number, amount paid, remaining balance, and tenant signature confirmation.
Should Have	US-10	Move-Out Clearance & Contract Deposit Reconciliation	**As a** student boarder, **I want** to check my outstanding balance and payment history on my portal, **so that** my advance deposit can be accurately applied to my final bill when I move out	Given a student is moving out at the end of the semester, when the caretaker initiates the check-out process, then the system cross-checks all historical unpaid balances and contract terms, automatically deducting owed amounts from the

Priority	ID	Epic	User Story / Requirement	Basic Acceptance Criteria
				1-month deposit before issuing final clearance.
Must Have	**US-11**	Role-Based Caretaker Cash Entry vs. Landlord Ledger Review	**As a** caretaker, **I want** a restricted transaction entry portal that only permits cash collection and OR logging, **so that** sensitive landlord financial configurations remain secure.	**Given** a caretaker logs into the application, **when** they navigate to the payment portal, **then** the system grants access only to cash collection and OR logging interfaces, restricting access to system-wide ledger configurations reserved for the landlord.
Must Have	**US-12**	In-Person Digital Acknowledgment / Signature Logging	**As a** caretaker, **I want** to capture a digital signature or manual payment acknowledgement flag when issuing an Official Receipt (OR), **so that** on-site payments are recorded with mutual proof.	**Given** a caretaker receives an in-person cash payment, **when** they record the OR details and present the receipt confirmation screen, **then** the system requires a tenant digital signature or acknowledgement check mark before finalizing the transaction entry into the ledger.
Must Have	**US-13**	Evasion Flagging	**As a** landlord, **I want** the system to flag past tenants who left with unsettled debts, **so that** they are	**Given** a tenant attempts to register or check out, **when** the system cross-checks their

Priority	ID	Epic	User Story / Requirement	Basic Acceptance Criteria
			prevented from re-registering or renting bedspaces in the future.	account history against unpaid balance records, **then** it automatically flags any prior unpaid debts ("gansi") to prevent unresolved lease departures or re-registration.

Prioritize your user stories using High/Medium/Low or Must Have/Should Have/Could Have. Consider client value, urgency, dependencies, risk, and feasibility.

Exercise 7 — Select and Justify Your Agile Approach

Choose Scrum, Kanban, or a Hybrid approach. Justify the selection based on the actual project rather than familiarity alone.

SELECTED APPROACH: Scrum Framework

- **Nature of the application:** Web-based dorm management system (RenTahanan) with distinct modules (bedspace tracking, fixed-rate rent billing, OR/ledger tracking, and tenant clearance).
- **Stability of client requirements:** Semi-fluid; core business rules (flat-rate rent, paper OR logging, 50+ beds) are clear, but user interface preferences (caretaker mobile input vs owner financial audits) evolve as prototypes are tested.
- **Frequency of client feedback:** Bi-weekly (at the end of each 2-week Sprint) to demo working increments to the caretaker and owner.
- **Team size and project duration:** Small student development team (3-5 members) working over an academic term (~8 to 12 weeks).
- **Nature of development tasks:** Feature-based modular development using the PERN stack (PostgreSQL, Express, React, Node.js).
- **Need for fixed iterations:** High; structured 2-week Sprints ensure steady progress and measurable deliverables before final project submission.

- **Expected frequency of changes:** Moderate; changes occur primarily during Sprint reviews when non-tech caretakers test the real-world flow of logging physical cash receipts.
- **How progress will be visualized and monitored:** Github Repositories / Trello Kanban board tracking Sprint Backlog items (To Do, In Progress, Code Review, Done) alongside daily standups.

Required output: One paragraph explaining why the selected approach fits your application, client, and team.

- We selected **Scrum** because its structured 2-week Sprints provide the predictability our student development team needs to build RenTahan within a strict academic timeline while offering the flexibility required by Evelyn Lady's Dormitory. Because our client is transitioning from paper logbooks, requirements regarding caretaker interface design and paper Official Receipt (OR) logging workflows were refined through hands-on testing rather than fixed upfront. Delivering working increments, starting with Must-Have features like visual bedspace management and fixed-rate rent billing in early Sprints allows the caretaker and owner to evaluate real-world usability frequently. This ensures potential misalignments in payment tracking or role permissions are caught and corrected early without disrupting the overall database schema or project schedule.

Exercise 8 — Plan the First Sprint / Iteration
Sprint / Iteration Duration: ***2 weeks***

Sprint Goal: *Establish the core foundational system for RenTahanan by enabling real-time bedspace occupancy tracking and digital payment/Official Receipt (OR) logging for caretakers to prevent missed debts.*

Select only backlog items needed to accomplish the Sprint/Iteration Goal.

ID	Selected User Story	Priority	Reason for Inclusion
US-01	**As a** landlord, **I want** to view my property as color-coded room/bed cards, **so that** I can instantly see which beds are occupied, overdue, or vacant at a glance.	Must Have	Core foundational features needed to digitize the physical notebook logbook and give immediate visibility into vacant, occupied, and full beds (single, double, quad rooms).
US-O2	**As a** caretaker, **I want** to assign new boarders to specific bed spaces and register their contract details, **so that** I can prevent double-bookings and maintain accurate bed occupancy records	Must Have	Essential for registering new borders, locking in 1-month advance rent and 1-month deposits, and assigning them to specific beds without double-booking.
US-04	As a landlord, I want the system to generate fixed monthly rent statements with bundled utility allowances, so that I can send consistent monthly bills without calculating electric or water meters.	Must Have	Establishes the automated monthly billing engine using fixed rates and bundled utilities (no submeters) so statements are generated accurately
US-07	**As a** caretaker, **I want** to input cash payments along with their Official Receipt (OR)	Must Have	Directly solves the primary business pain point of "gansi" (revenue loss) by

ID	Selected User Story	Priority	Reason for Inclusion
	numbers, **so that** every payment is digitally logged with proof.		allowing caretakers to enter cash payments with OR numbers into a digital ledger.
US-08	**As a** landlord, **I want** the system to automatically calculate and flag unpaid rent balances, **so that** I can track remaining debts and prevent revenue loss.	Must Have	Automatically calculates remaining tenant debts and flags overdue accounts, replacing vulnerable manual paper tracking.

Break the selected stories into manageable development tasks.

User Story	Development Task	Responsible Member(s)	Expected Output
US-01	Design database tables for rooms and beds; create frontend color-coded occupancy card UI.	Developer 1 (Frontend / DB)	Functional dashboard displaying room cards with live vacant/occupied badges.
US-02	Create tenant registration form and bed space assignment modal; link contract deposit data.	Developer 2 (Backend / UI)	Registration form that updates bed status to "Occupied" and saves tenant details.
US-04	Implement flat-rate monthly billing logic and bundled utility	Developer 3 (Backend)	Automated invoice generator applying

User Story	Development Task	Responsible Member(s)	Expected Output
	allowance config (phones, laptops free).		fixed monthly rent rates per tenant.
US-07	Build payment entry interface for caretakers to log cash amounts, payment dates, and OR numbers	Developer 1 & 2	Form that records cash transactions and attaches OR numbers to tenant ledgers.
US-08	Develop automated balance calculation algorithms and overdue tenant highlight indicators.	Developer 3 (Backend)	Real-time balance calculation logic that flags unpaid accounts on the dashboard.

Exercise 9 — Create the Initial Definition of Done

Develop at least 5 conditions that must be satisfied before work is marked Done.

- **Acceptance Criteria Met:** The completed feature fully satisfies all written Given–When–Then acceptance criteria for the user story (e.g., payment entry correctly logs Official Receipt numbers without calculation errors).
- **Database Integrity & Ledger Accuracy:** All backend operations update the PostgreSQL database correctly, ensuring ledger calculations (balances, advance deposits, and fixed monthly rents) are mathematically accurate.
- **Role-Based Access Verification:** The feature is verified to operate correctly under the appropriate user role (e.g., caretakers can log cash payments/ORs, while tenants only have read-only portal access).
- **Usability & Interface Testing:** The user interface is tested and confirmed to be intuitive and functional on web browsers, ensuring non-tech-savvy caretakers can complete tasks easily without confusion.

- **Code Review & Deployment:** The source code has been peer-reviewed, free of critical bugs, and successfully merged into the main development branch without breaking existing features.

Exercise 10 — Establish Team Workflow and Responsibilities

At minimum, use: BACKLOG → TO DO → IN PROGRESS → REVIEW/TESTING → DONE.

If using Kanban or Hybrid, specify an appropriate Work-in-Progress (WIP) limit.

- **BACKLOG:** All unprioritized user stories and system features for RenTahan.
- **TO DO:** Selected user stories committed for the current 2-week Sprint iteration
- **IN PROGRESS:** Tasks currently being implemented by assigned developers.
- **REVIEW / TESTING:** Completed features undergoing peer code review, database testing, and usability checks.
- **DONE:** Features fully verified against all conditions in the Definition of Done.

Responsibility	**Member(s)**
Client communication	Jacuba
Requirements coordination	Malinao
UI/UX	Jacuba, Malinao
Frontend development	Jacuba, Malinao
Backend development	Papolonias
Database	Papolonias
Testing / QA	Papolonias
Documentation	Jacuba, Malinao
Repository / version control coordination	Papolonias

Exercise 11 — Mini-Retrospective and Risk Planning

Identify at least 5 likely risks and agree on how the team will respond.

Potential Risk	**Possible Effect**	**Team Response**
Caretaker finding digital entry complex or slow	Caretaker bypasses system and reverts to physical paper logbooks, leading to incomplete payment records.	Conduct usability testing with simplified, large touch-target mobile payment forms (quick

Potential Risk	Possible Effect	Team Response
		OR logging) and provide a concise visual user guide.
Unrecorded or delayed cash payment entries	Disparencies between physical cash on hand and backend ledger balances, causing revenue loss.	Implement mandatory OR reference filed validation and automated unpaid balance alerts to highlight uncollected rent.
Database schema mismatch during PERN integration	Broken REST API endpoints between Node.js/Express backend and React frontend, halting development progress.	Enforce strict API data contracts and require backend developer to test PostgreSQL queries prior to frontend integration
Miscalculated advance deposits during student move-out	Financial conflict or revenue loss during tenant check-out due to unverified past debts.	Implement automated deposit deduction reconciliation logic that flags historical unpaid balances before clearance
Scope creep from additional client feature requests	Delay in completing core "Must Have" deliverables within the strict academic 2-week Sprint cycle.	Strictly enforce MoSCoW prioritization, placing non-essential requests into the Product Backlog for future Sprints after MVP deployment.

START — What should the team begin doing?

- Begin conducting quickly daily 5-minute async standups via group chat to flag integration blockers (PostgreSQL API routes, React state management) earlier in the 2-week Sprint Cycle.

STOP — What should the team avoid?

- Stop pushing unreviewed code directly to the main Git repository branch without conducting peer testing or testing database transaction edge cases.

CONTINUE — What is working and should continue?

- Continue using the clear division of team roles to ensure fast parallel to development.

One concrete, measurable team agreement:

- Before any new code is added to the main project, at least one other groupmate must test it and confirm it meets all the Given-When-Then criteria.

III. Required Outputs for This Week

5. Development Life-Cycle Diagram
6. Sequential vs. Iterative Development Analysis
7. At least 3 Epics
8. At least 10 User Stories
9. Acceptance Criteria for the User Stories
10. Initial Prioritized Product Backlog
11. Agile Method Selection and Justification
12. First Sprint / Iteration Plan
13. Initial Definition of Done
14. Team Workflow and Responsibilities
15. Mini-Retrospective and Risk Plan

Submission deadline: September 25, 2026, at 12:00 midnight.

Submit the complete Week 3 group assessment by the deadline specified above.

IV. Presentation Next Week

Each group will present its completed output during the lecture class next week.

The presentation should demonstrate how Agile will be applied to the group's actual application and client. Each group should be prepared to explain:

- The client problem and proposed application
- How the application will be developed iteratively
- Major Epics and User Stories

- How the Product Backlog was prioritized
- Why the selected Agile approach is appropriate
- The first Sprint/Iteration Goal and selected work
- The team's Definition of Done
- How work and responsibilities will be managed
- Major project/team risks and planned responses

Every member should understand the plan and be prepared to answer questions during the presentation.

V. Assessment Plan

Assessment	**Mode**	**Evidence / Expected Output**	**Official Category**
Agile Process Diagram	Group	Visual representation of how the team will move from requirement discovery through iterative development, client feedback, and final release.	Assignments / Lab Activities
Initial Product Backlog	Group	Prioritized project features/user stories with basic acceptance criteria.	Team Project
Methodology Justification	Group	Short explanation of why the chosen Agile approach fits the project, client, and team.	Team Project
Sprint / Iteration Plan	Group	Sprint Goal, selected backlog items, tasks, responsibilities, and expected outputs.	Team Project

Assessment	Mode	Evidence / Expected Output	Official Category
Definition of Done	Group	Agreed conditions that must be satisfied before work is considered complete.	Team Project
Risk & Mini-Retrospective	Group	Identified risks, planned responses, and Start–Stop–Continue agreements.	Assignments / Lab Activities

VI. Final Reminder

This is an applied assessment. Do not simply define Scrum, Kanban, Product Backlogs, Sprints, or user stories. Use these concepts to plan the actual application that your group will develop for its client.

Complete and submit the assessment by September 25, 2026 at 12:00 midnight, and be ready to present and defend your Agile Project Plan during the lecture class next week.